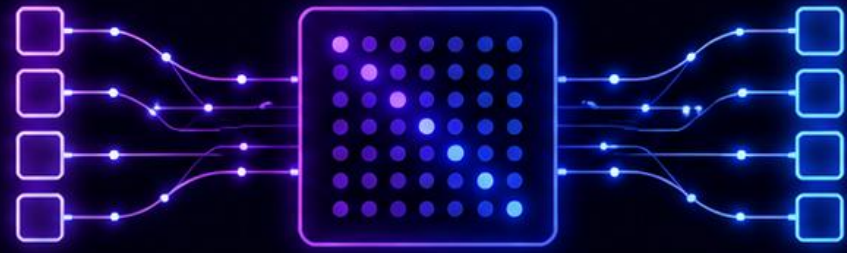
UNDERSTANDING

TRANSFORMERS

A Deep Dive into the Architecture
That Revolutionized Deep Learning



2026 SPRING SEMESTER

PRESENTED BY

Serag Khaled • Hussein Mostafa • Mohannad Hassan

SUPERVISED BY

Dr. Mohsen Rashwan & Dr. Mohamed Abdelghany

Why Transformers Over RNNs/LSTMs?

Transformers address the fundamental limitations of RNNs/LSTMs
and enable state-of-the-art performance at scale.

Limitation of RNNs/LSTMs	How Transformers Are Better
Sequential Processing RNNs process tokens one at a time, limiting speed and parallelization.	**Full Parallelization** Transformers process all tokens simultaneously, enabling massive speedups and better hardware utilization.
Struggles with Long-Range Dependencies Information fades over long sequences (vanishing gradients).	**Direct Attention to All Tokens** Self-attention connects every token to every other token regardless of distance.
Less Efficient at Scale RNNs have inherently sequential computations and lower throughput.	**Highly Scalable** Better training efficiency, shorter time-to-train, and scales to billions of parameters.
More Data & Memory Hungry Require more memory for long sequences and have lower data efficiency.	**More Data Efficient** Attention mechanisms make better use of data, requiring less data to generalize.
Harder to Optimize RNNs are harder to train and tune effectively.	**Easier to Train & Optimize** Stable gradients, better optimization, and easier experimentation.
Limited in Multi-Modal & Complex Tasks Harder to adapt to diverse tasks (vision, text, audio, code).	**Versatile Across Domains** Used in NLP, Vision, Speech, Robotics, Code, and more with the same core architecture.

The Result

Transformers power the most advanced models today—GPT, BERT, PaLM, Claude, Gemini, and more.
They set the new standard for performance, scalability, and flexibility.

What Are Transformers?

Built for Parallelism

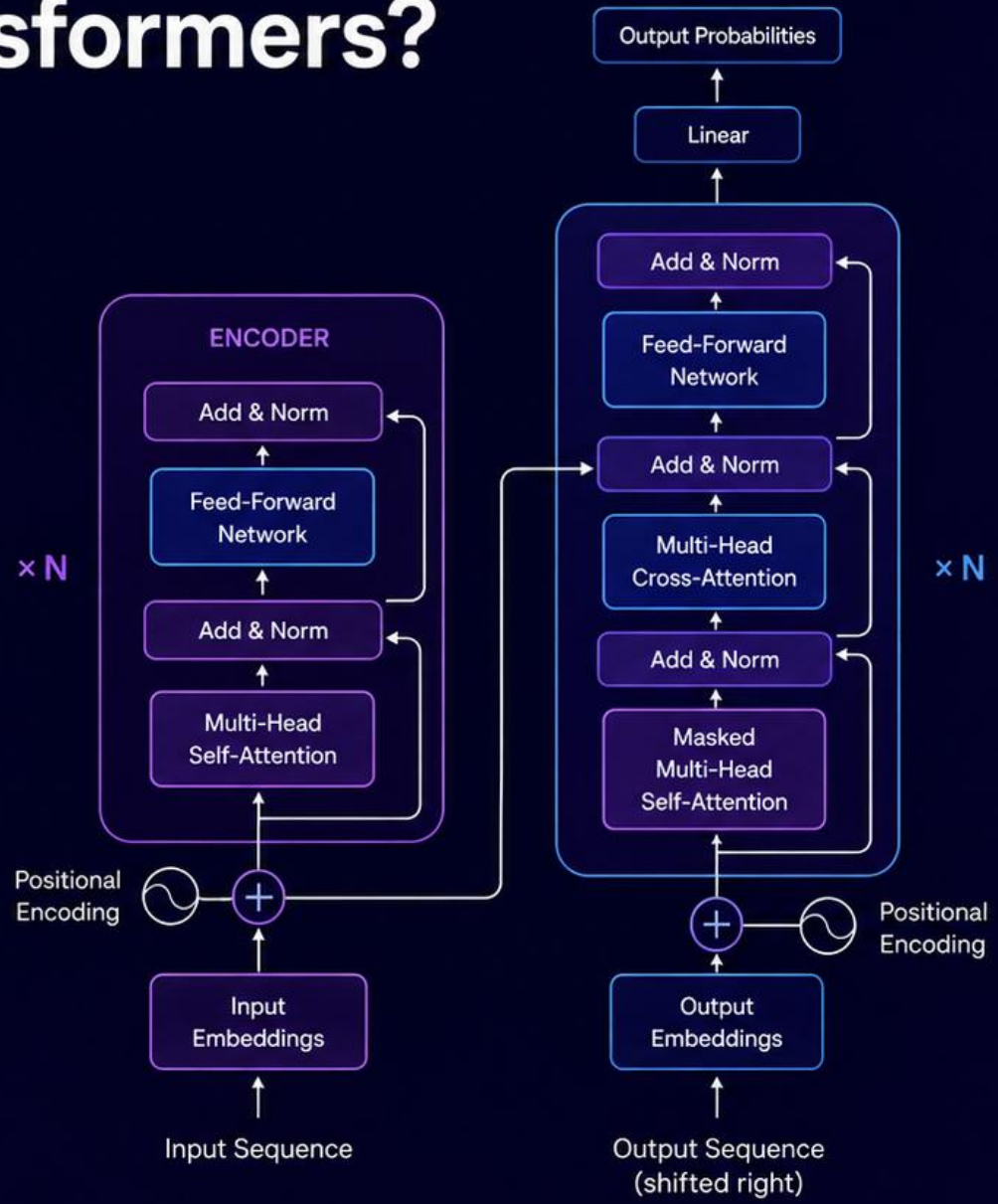
Process entire sequences at once for faster training and inference.

No Recurrence

Rely on attention mechanisms instead of RNNs or LSTMs to model dependencies.

Capture Long-Range Dependencies

Attention connects every token to every other token.



Scalable & Efficient

Designed to handle long sequences with better scalability.

Versatile Foundation

Foundation of powerful models like BERT, GPT, and T5.

Attention is the Key

Allows the model to focus on the most relevant parts of the sequence.

Key Components of the Transformer

1 Positional Encoding

Injects sequence order information into token embeddings since there is no recurrence.

2 Multi-Head Self-Attention

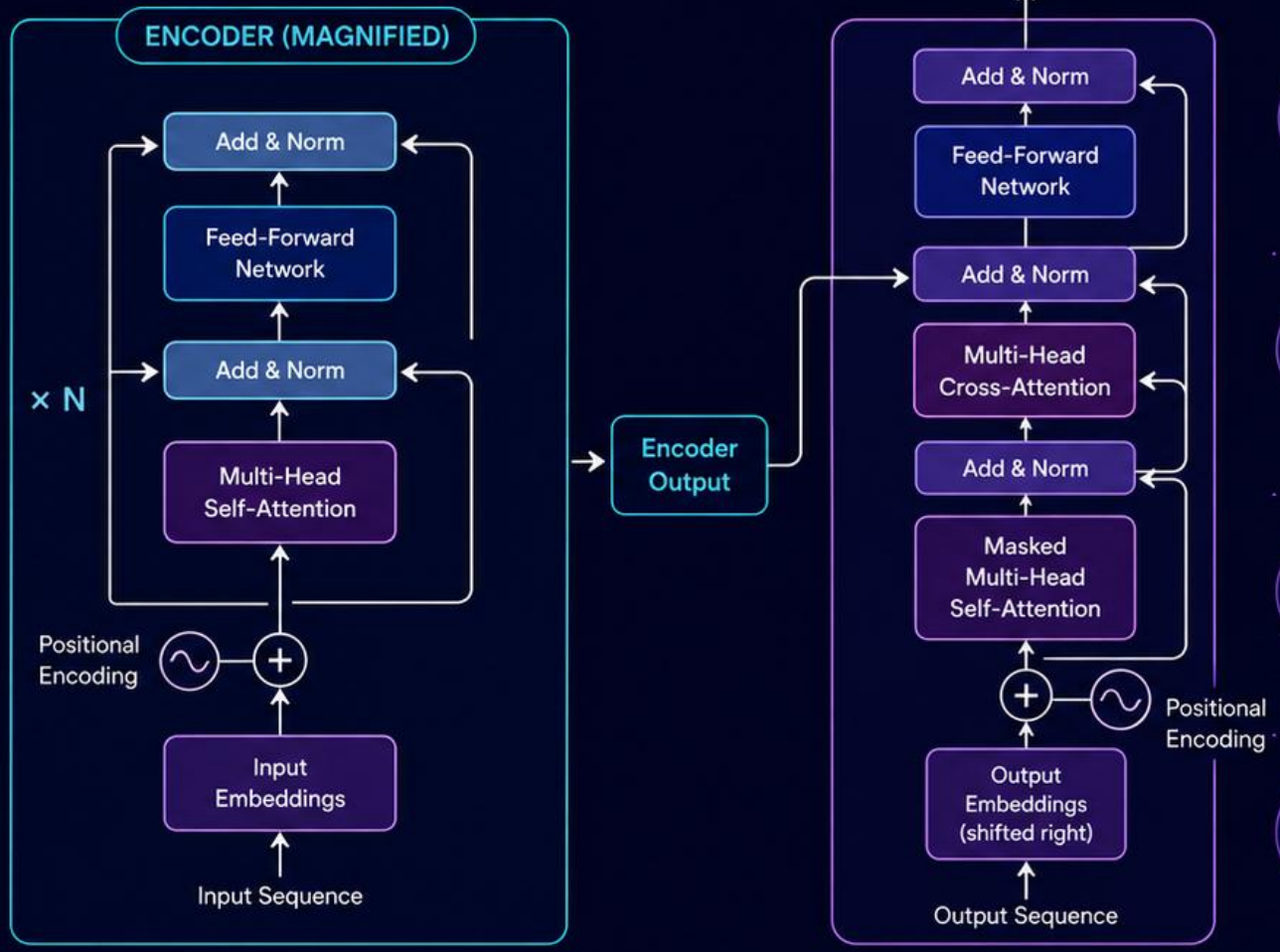
Allows the model to attend to different parts of the sequence in parallel. Captures multiple relationships at once.

3 Feed-Forward Network (FFN)

Applies a fully connected network to each token independently.
Adds non-linearity and feature transformation.

4 Add & Layer Normalization

- Residual (skip) connections preserve information.
- LayerNorm stabilizes and speeds up training.



Built for Parallelism

Process entire sequences at once for faster training and inference.

No Recurrence

Rely on attention mechanisms instead of RNNs or LSTMs to model dependencies.

Capture Long-Range Dependencies

Attention connects every token to every other token.

Versatile Foundation

Foundation of powerful models like BERT, GPT, and T5.

EMBEDDING & POSITIONAL EMBEDDING

In Transformers

1 What is Token Embedding?

- Converts each token (word/subword) into a dense vector representation.
- Captures semantic meaning of tokens in a continuous vector space.

2 What is Positional Embedding?

- Adds information about the position of tokens in the sequence.
- Since Transformers have no recurrence or convolution, this provides order awareness.

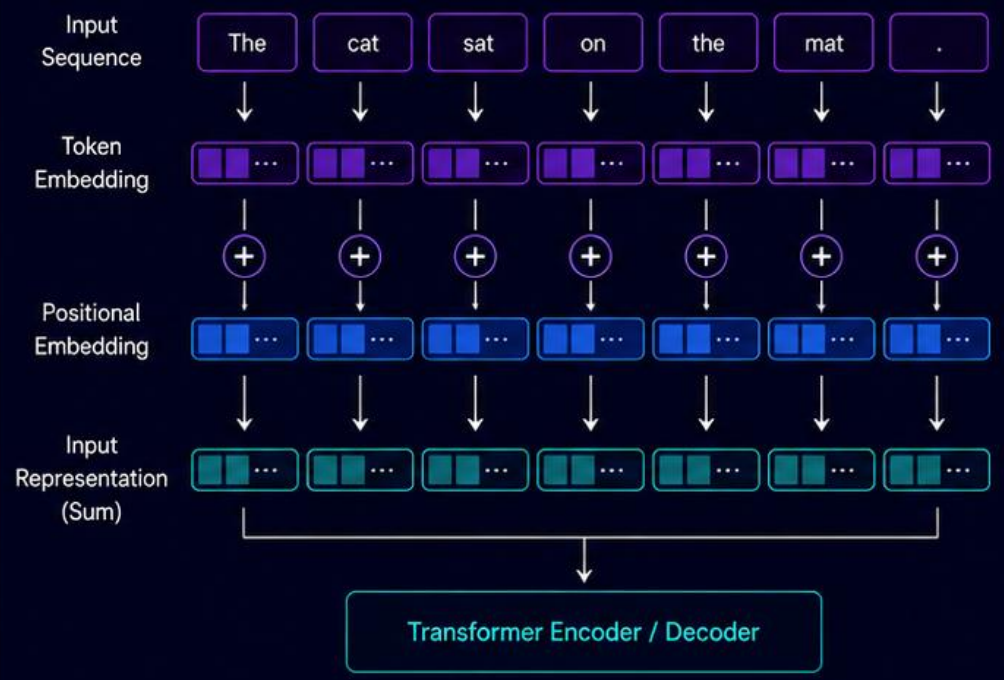
3 How They Work Together

- Token Embedding + Positional Embedding = Input Representation
- This combined representation is fed into the Transformer Encoder/Decoder.

4 Why They Matter?

- Enable the model to understand both "what" the token is and "where" it is.
- Essential for capturing the structure and meaning of sequences.

From Tokens to Input Representation



Example (d = 4)

Token Embedding for "cat"

0.21	-0.31	0.76	0.11

Positional Embedding for position 2

0.01	0.23	-0.17	0.32

Input Representation (Sum)

0.22	-0.08	0.59	0.43

Positional embeddings can be learned (trainable) or fixed (sinusoidal).

Key Benefits

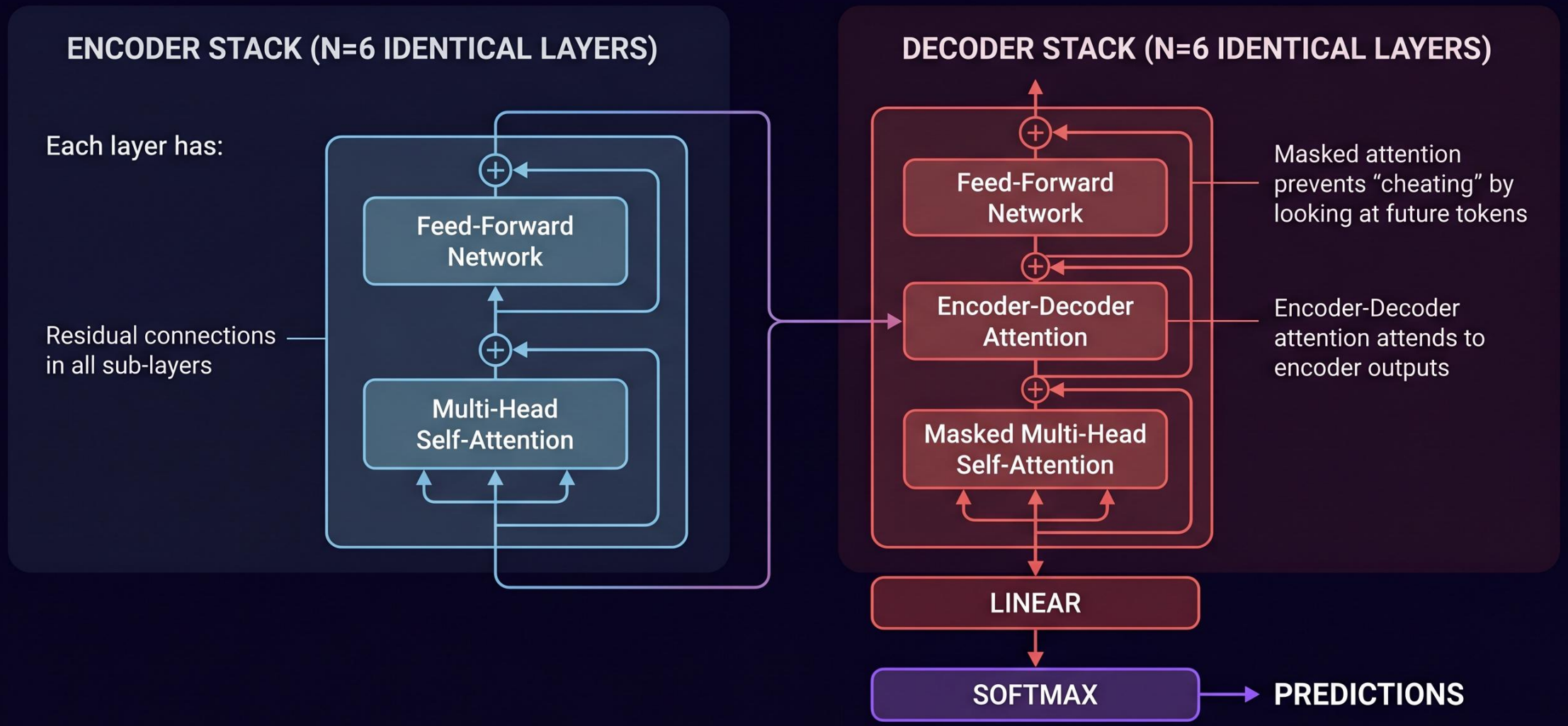
Meaning + Order
Combines semantic meaning with position information.

Better Understanding
Helps the model understand context and relationships accurately.

Generalization
Learned embeddings adapt to tasks and datasets effectively.

Foundation for Learning
These embeddings are the foundation of all Transformer representations.

Transformer Architecture: Encoder & Decoder



Self-Attention Mechanism Explained

1. Core Idea

Computes a weighted representation of each token based on its relationship to all other tokens in the sequence.

2. Key Vectors

Uses three key vectors for each token: Query (Q), Key (K), and Value (V).

3. Formula

Attention(Q, K, V) = softmax

$$\left(QK^T / \sqrt{d_k}\right) V$$

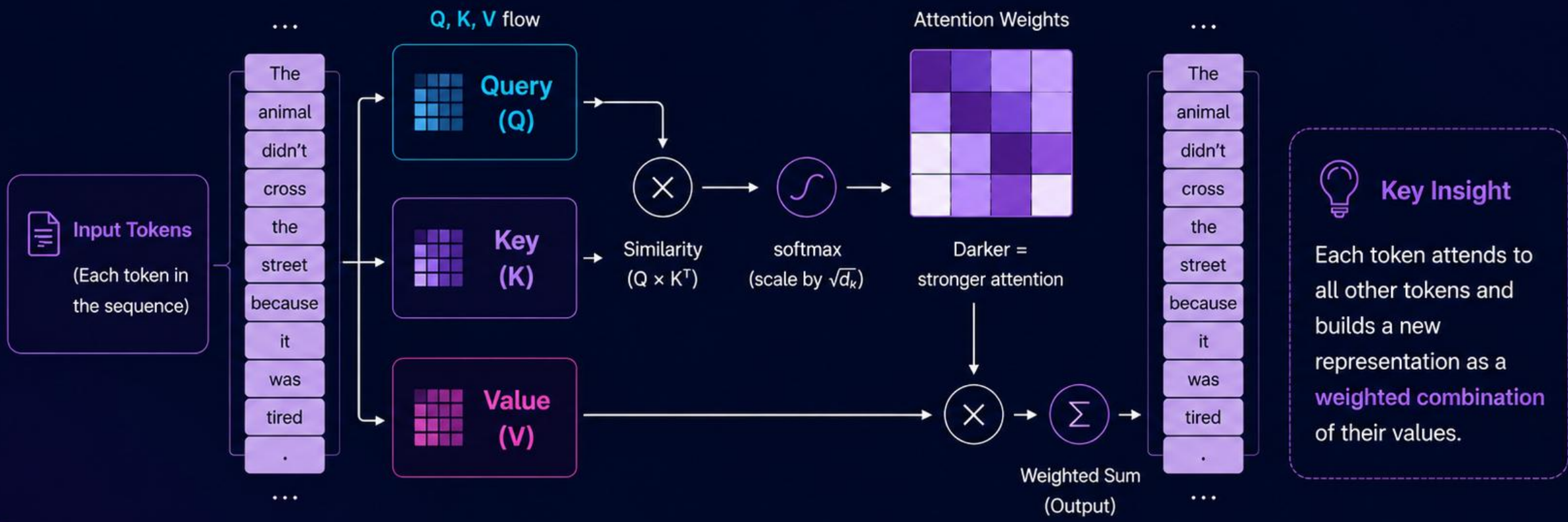
4. Captures Dependencies

Captures both local and long-range dependencies in a single step.

5. Example

The animal didn't cross the street because it was tired. Self-attention identifies it as referring to animal.

Self-Attention
How It Works



Key Insight

Each token attends to all other tokens and builds a new representation as a **weighted combination** of their values.

MULTI-HEAD ATTENTION

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. Each head learns unique patterns and relationships in parallel.

1 The Idea

Instead of performing a single attention operation, we linearly project the inputs *h h* times with different learned projections and perform attention in parallel.

2 How It Works

- Project the inputs into *h* different subspaces.
- Perform scaled dot-product attention in parallel for each head.
- Concatenate the outputs and project them back to the model dimension.

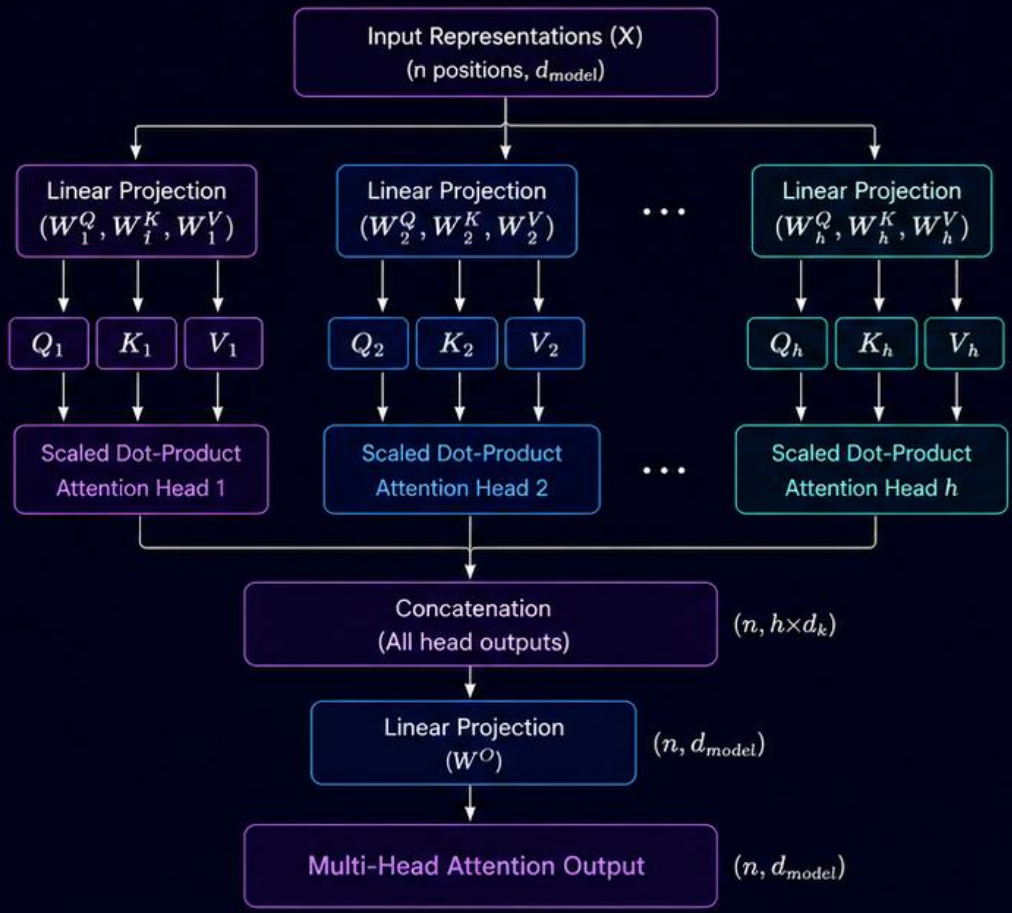
3 Why Multi-Head?

- Captures diverse types of relationships.
- Attends to information from multiple positions and representation subspaces.
- More expressive and powerful.

4 Where It's Used

Used in both the Encoder and Decoder self-attention layers of Transformers.

Multi-Head Attention Mechanism



Inside Each Attention Head

For head i:

$$\text{Attention}_i(Q_i, K_i, V_i) = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right) V_i$$

where:

- $Q_i = XW_i^Q$ (Queries)
- $K_i = XW_i^K$ (Keys)
- $V_i = XW_i^V$ (Values)
- d_k = dimension of each head

Dimensions

X	$n \times d_{model}$
W_i^Q, W_i^K, W_i^V	$d_{model} \times d_k$
Head Output	$n \times d_k$
Concatenated Output	$n \times (h \times d_k)$
W^O	$h \times d_k \times d_{model}$
Final Output	$n \times d_{model}$

n = number of positions (sequence length)
d_{model} = model dimension
h = number of heads
d_k = dimension per head ($d_k = d_{model}/h$)

Key Advantages

Captures Diverse Patterns

Each head learns different patterns (syntactic, semantic, positional, etc.).

More Expressive

Aggregating multiple perspectives makes the model more powerful and expressive.

Better Generalization

Learns richer representations leading to improved performance across tasks.

Efficient & Parallelizable

All heads are computed in parallel, making it efficient on modern hardware.

Masked Multi-Head Attention

1 What is it?

A variant of multi-head attention that prevents each position from attending to future positions.
Ensures causality in autoregressive generation.

2 How it Works

① Input embeddings are projected into Q (Query), K (Key), V (Value).
② Scaled dot-product attention is computed.
③ A look-ahead mask is applied to the attention scores.
④ Outputs from all heads are concatenated and passed through a linear layer.

3 Core Equation

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

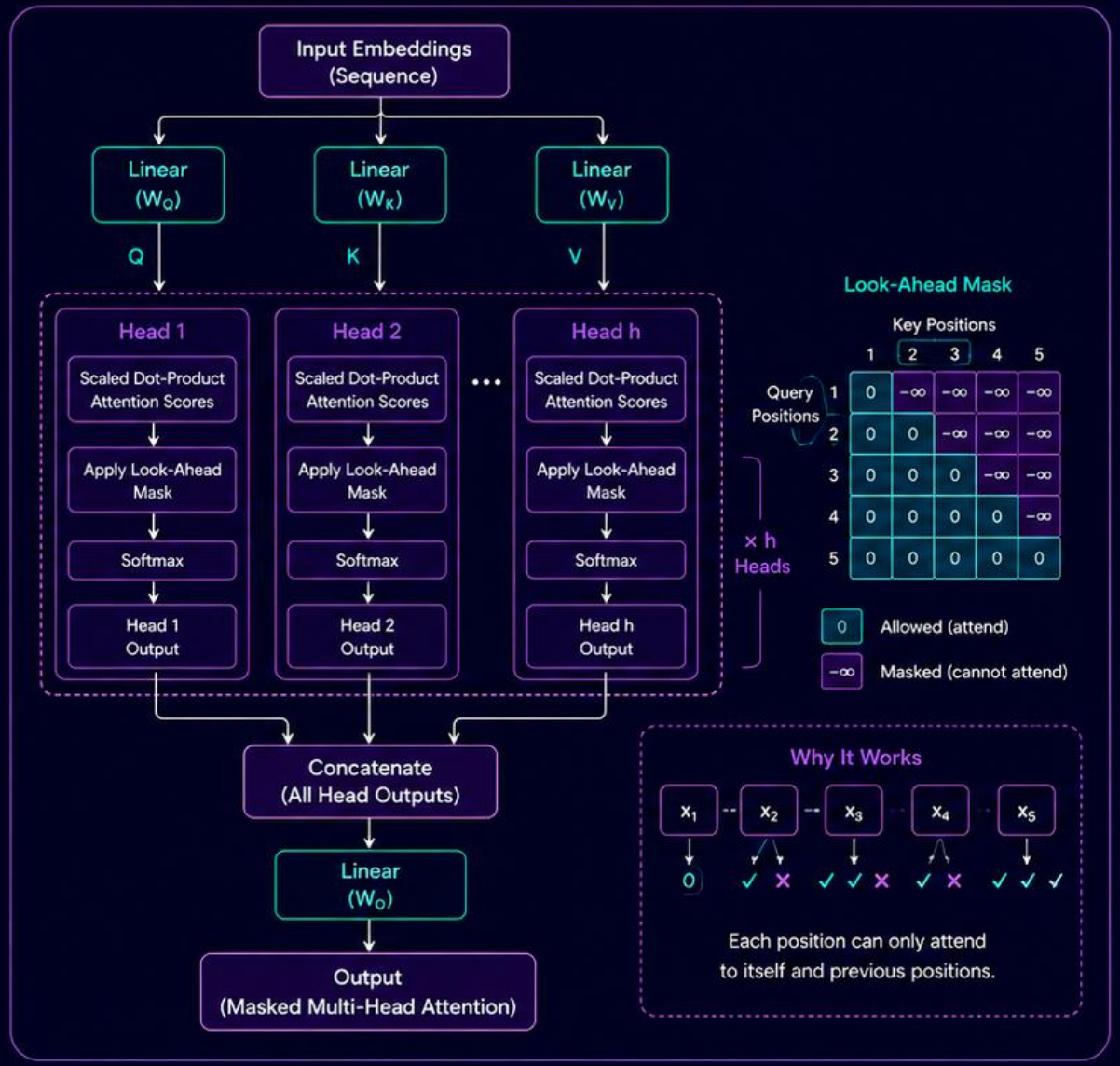
where M is the look-ahead mask
(0 for allowed positions, $-\infty$ for masked).

4 Why Masking?

- Prevents attention to future tokens.
- Maintains the autoregressive property.
- Essential for sequence generation.

5 Where it's Used

Primarily used in the Decoder Self-Attention block of Transformers.



Preserves Causality

Ensures the model cannot peek into future tokens during training or inference.

Better Generation

Produces coherent and consistent sequences token by token.

Stable Training

Preventing information leak leads to more stable and effective training.

Scalable & Efficient

Masking is simple yet highly effective and scales well with sequence length.

Key Advantages

Maintains Autoregression

Guarantees the model only uses past and present information.

Improves Performance

Helps the model learn accurate dependencies for generation tasks.

Enables Parallel Training

All positions are processed in parallel while respecting the causal order.

Essential for Decoders

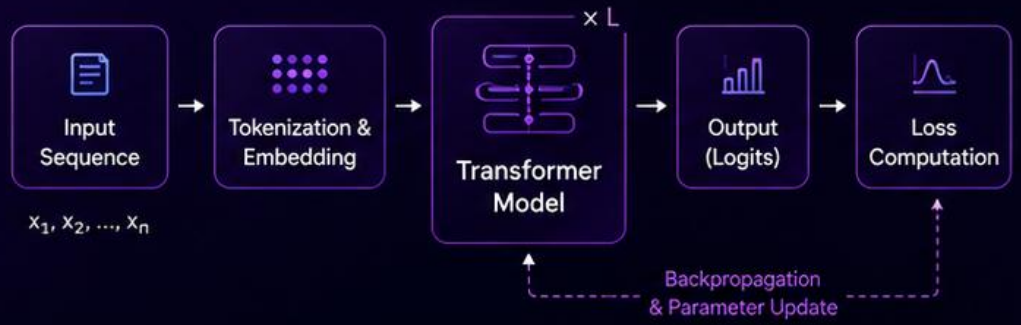
A core component for autoregressive models like GPT, LLaMA, etc.

TRAINING & INFERENCE

OF TRANSFORMERS

TRAINING

Learn model parameters from large datasets



Objective

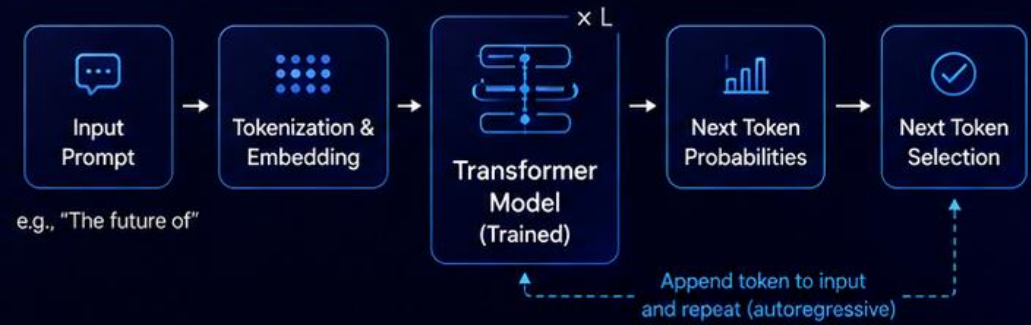
Minimize loss (e.g., Cross-Entropy) between predicted and target tokens

What's Learned?

Model learns the parameters (weights) of attention and feed-forward layers to predict next tokens accurately.

INFERENCE

Use learned parameters to generate predictions



How It Works

The model predicts the next token based on the previous tokens. Repeat until stopping condition.

Key Point

No parameter updates. The model only performs forward passes.

SUMMARY

Training teaches the model. Inference puts it to use.

Training

- Uses target labels
- Updates parameters
- High compute

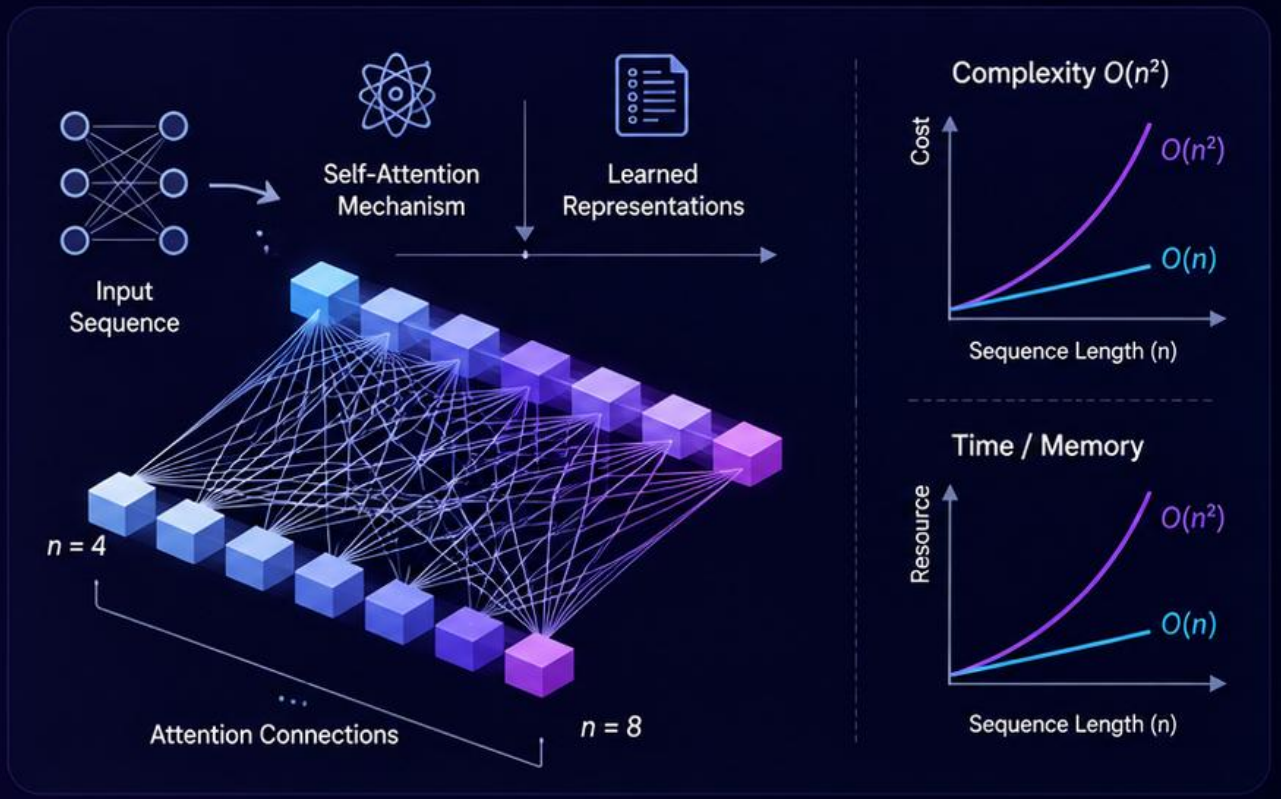
Inference

- No labels needed
- No parameter updates
- Optimized for speed

Same Model, Two Phases

Training builds knowledge. Inference generates intelligence.

Limitations & Failure Cases



1 Quadratic Complexity

Self-attention scales as (n^2) with sequence length — expensive for long sequences.

2 Data Hunger

Transformers require large amounts of training data; may underperform on small datasets.

3 High Computational Cost

Training large models needs significant GPU/TPU resources.

4 Interpretability

Attention weights don't always provide clear explanations.

5 Not Ideal for Very Long Contexts

Resource-constrained environments or tasks with limited labeled data.

Transformers Attention Quiz

Test your understanding of the Attention Mechanism in Transformers

Q1 What is the main purpose of the attention mechanism in Transformers?

- A Reduce dataset size
- B Focus on relevant parts of the sequence
- C Replace embeddings
- D Increase model depth

Q2 In self-attention, each token compares itself with:

- A Only the previous token
- B Only the next token
- C All other tokens in the sequence
- D The output layer only

Q3 Which components are used to compute attention scores?

- A Filters, Kernels, Biases
- B Queries, Keys, Values
- C Inputs, Outputs, Labels
- D Weights, Activations, Gradients

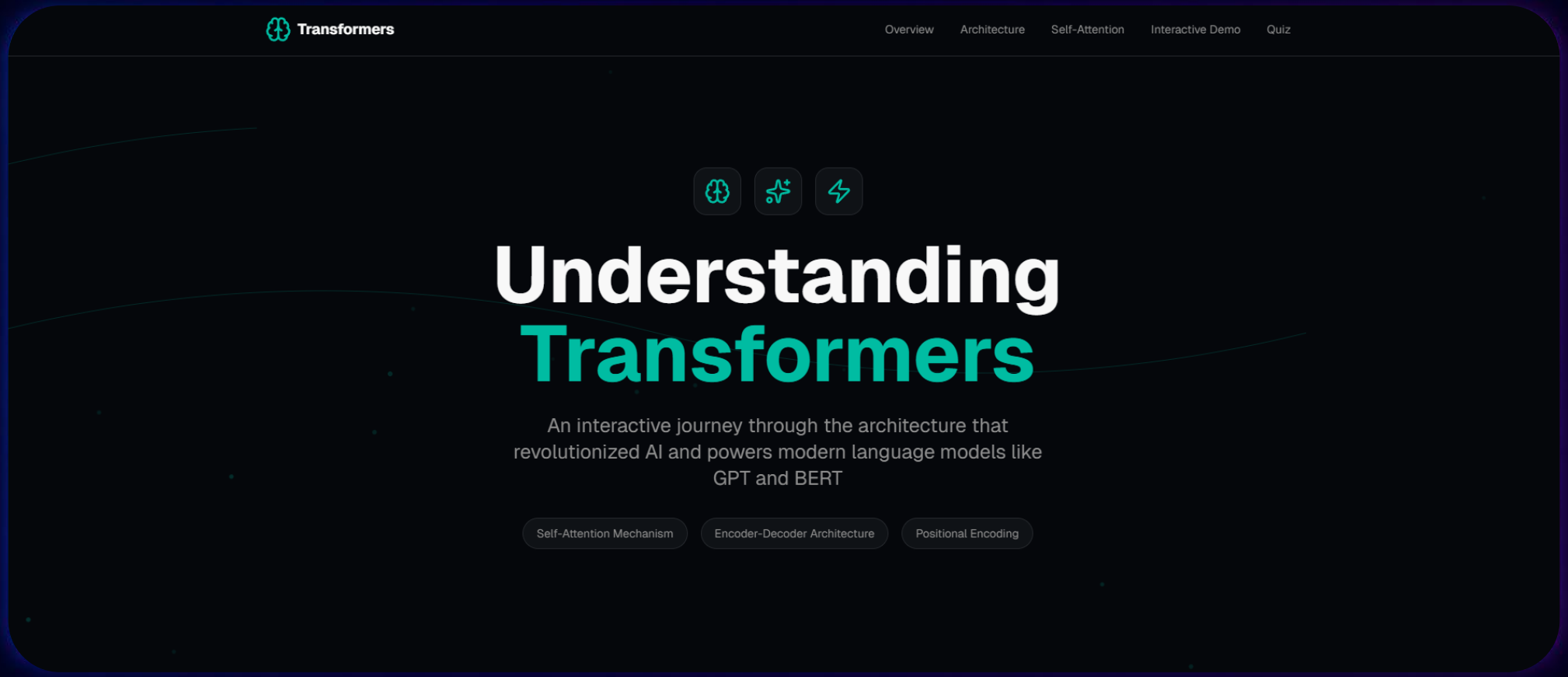
Q4 Why are Transformers faster to train than RNNs/LSTMs?

- A They use fewer parameters
- B They avoid embeddings
- C They process sequences in parallel
- D They require smaller datasets

Q5 What problem do Transformers handle better than traditional RNNs?

- A Image compression
- B Vanishing gradients only
- C Long-range dependencies
- D Feature normalization

For More Interactive Content and Questions Click This:



THANK YOU!